AWS Cloud Engineering Project

# EC2 Instance Scheduler

Automated Start / Stop Using AWS Lambda & Amazon EventBridge

David Koduah · Cloud Engineer Portfolio · March 2026

This project automates the start and stop of an AWS EC2 instance on a fixed business-hours schedule. Two Python Lambda functions are triggered by Amazon EventBridge cron rules — starting the instance at 09:00 UTC and stopping it at 17:00 UTC, Monday to Friday. This eliminates manual intervention and reduces compute costs by over 70% by ensuring the instance only runs during productive hours.

AWS Lambda | EventBridge | Amazon EC2 | Python 3.14 | Boto3 1.42.78

| Cost Saving | Functions | Schedule | Tests Passed |
|---|---|---|---|
| 70%+ | 2 | Mon–Fri 09:00–17:00 UTC | ✓ All Succeeded |
| compute reduction | start-ec2 + stop-ec2 | business hours only | 0 errors recorded |

## 1 · Architecture Overview

The solution is fully serverless and event-driven. Amazon EventBridge fires cron-based triggers at scheduled times, invoking the appropriate Lambda function. Each Lambda uses the Boto3 SDK to call the EC2 API and start or stop the target instance. IAM roles control what each function is permitted to do, following the principle of least privilege.

**Amazon EventBridge**

start-rule cron(0 9 ? * mon-fri *)
stop-rule cron(0 17 ? * mon-fri *)

**AWS Lambda**

start-ec2 (Python 3.14) Boto3 start_instances() stop-ec2 (Python 3.14) Boto3 stop_instances()

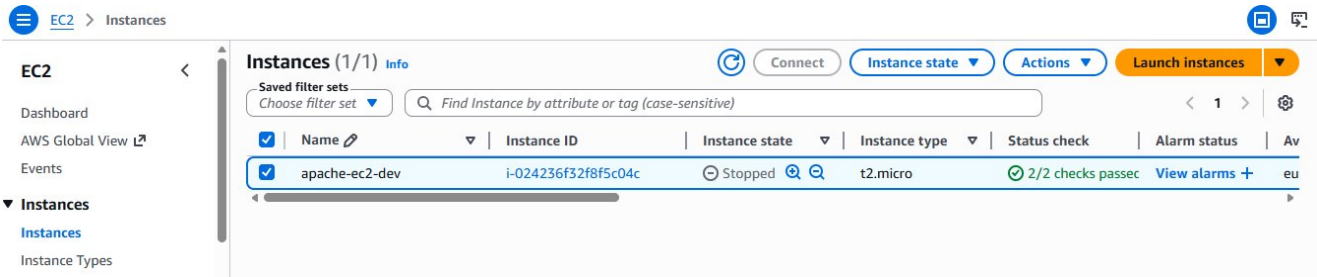
**Amazon EC2**

apache-ec2-dev t2.micro eu-west-2 (London)

IAM: AmazonEC2FullAccess + AWSLambdaBasicExecutionRole on each Lambda execution role. EventBridge granted invoke permission via resource-based policy (events.amazonaws.com).

## 2 · Starting Point — EC2 Instance in Stopped State

Before building the automation, the EC2 instance apache-ec2-dev was already created and sitting in a Stopped state. This is the baseline — the goal is to automate bringing it up at 9am and shutting it down at 5pm without any manual steps.




*apache-ec2-dev (t2.micro, eu-west-2) in Stopped state — 2/2 checks passed. This is the starting point before any Lambda automation was built.*

## 3 · Boto3 Documentation Research

Before writing the Lambda code, the Boto3 AWS SDK documentation was consulted to identify the correct API calls. Boto3 1.42.78 provides the EC2 Client with start_instances() and stop_instances() methods.

Boto3 1.42.78 documentation

Search

Feedback

Do you have a suggestion to

# EC2

## Client

ON THIS PAGE

- Client
  - EC2.Client
- Paginators
- Waiters
- Resources

class EC2.Client

A low-level client representing Amazon Elastic Compute Cloud (EC2)

You can access the features of Amazon Elastic Compute Cloud (Amazon EC2) programmatically. For more information, see the Amazon EC2 Developer Guide.

```
import boto3

client = boto3.client('ec2')
```

*Boto3 1.42.78 EC2 Client documentation — import boto3; client = boto3.client('ec2'). The entry point for all EC2 API calls via Python.*

Boto3 1.42.78 documentation

Search

Feedback

Do you have a suggestion to improve this website or boto3? Give us feedback.

Quickstart

A Sample Tutorial

Code Examples

Developer Guide

EC2 / Client / start_instances

# start_instances

EC2.Client.start_instances(**kwargs)

Starts an Amazon EBS-backed instance that you've previously stopped.

Instances that use Amazon EBS volumes as their root devices can be quickly stopped and started. When an instance is stopped, the compute resources are released and you are not billed for instance usage. However, your root partition Amazon EBS volume remains and continues to persist your data, and you are charged for Amazon EBS volume usage. You can restart your instance at any time. Every time you start your instance, Amazon EC2 charges a one-minute minimum for instance usage, and thereafter charges per second for instance usage.

Before stopping an instance, make sure it is in a state from which it can be restarted. Stopping an instance does not preserve data stored in RAM.

Performing this operation on an instance that uses an instance store as its root device returns an error.

If you attempt to start a T3 instance with `host` tenancy and the `unlimited` CPU credit option, the request fails. The `unlimited` CPU credit option is not supported on Dedicated Hosts. Before you start the instance, either change its CPU credit option to `standard`, or change its tenancy to `default` or `dedicated`.

*Boto3 start_instances() documentation — starts a previously stopped EBS-backed instance. Instances are charged per second after a one-minute minimum.*

Boto3 1.42.78 documentation

Search

Feedback

Do you have a suggestion to improve this website or boto3? Give us feedback.

Quickstart

A Sample Tutorial

Code Examples

Developer Guide

EC2 / Client / stop_instances

ON THIS PAGE

- EC2.Client.stop_instances()

# stop_instances

EC2.Client.stop_instances(**kwargs)

Stops an Amazon EBS-backed instance. You can restart your instance at any time using the StartInstances API. For more information, see Stop and start Amazon EC2 instances in the *Amazon EC2 User Guide*.

When you stop or hibernate an instance, we shut it down. By default, this includes a graceful operating system (OS) shutdown. To bypass the graceful shutdown, use the `skipOsShutdown` parameter; however, this might risk data integrity.

You can use the StopInstances operation together with the `Hibernate` parameter to hibernate an instance if the instance is enabled for hibernation and meets the hibernation prerequisites. Stopping an instance doesn't preserve data stored in RAM, while hibernation does. If hibernation fails, a normal shutdown occurs. For more information, see Hibernate your Amazon EC2 instance in the *Amazon EC2 User Guide*.

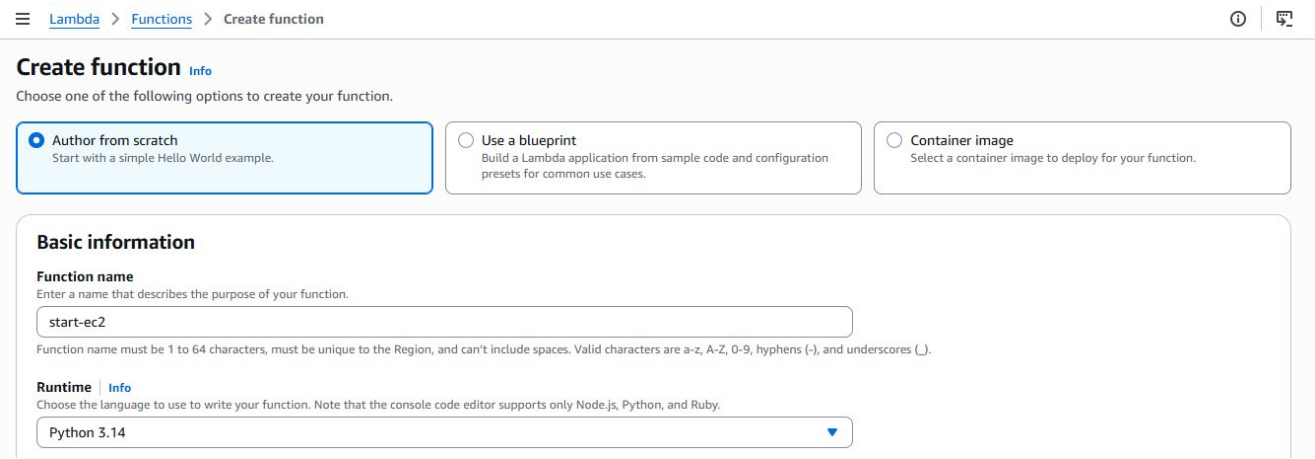
If your instance appears stuck in the `stopping` state, there might be an issue with the underlying host computer. You can use the StopInstances operation together with the Force parameter to force stop your instance. For more information, see Troubleshoot Amazon EC2 instance stop issues in the *Amazon EC2 User Guide*.

Stopping and hibernating an instance differs from rebooting or terminating it. For example, a

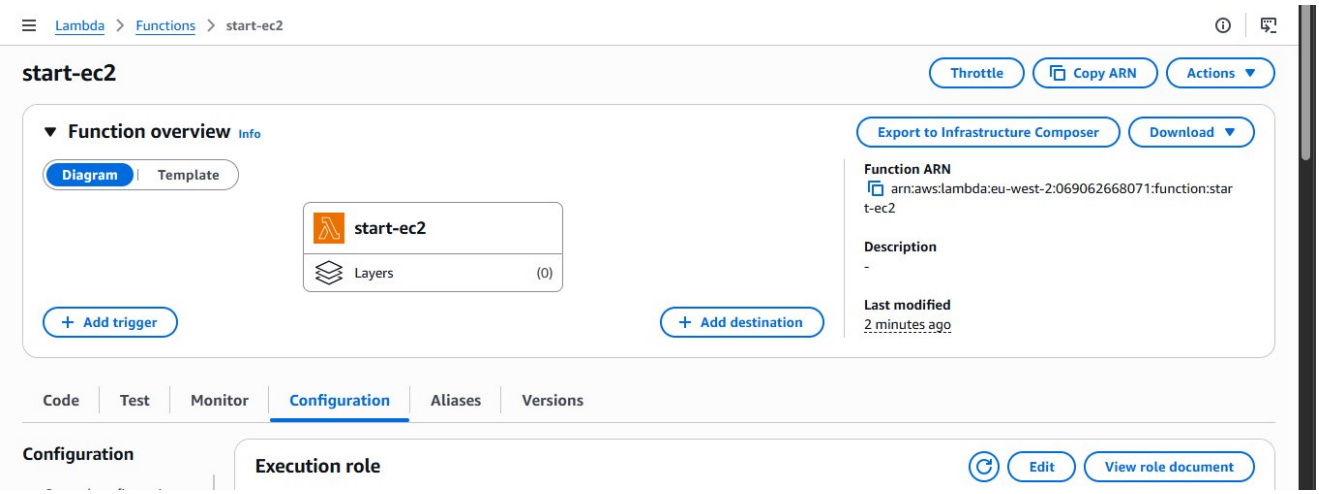
*Boto3 stop_instances() documentation — performs a graceful OS shutdown. Does not preserve data stored in RAM.*

# 4 · Creating the Lambda Functions

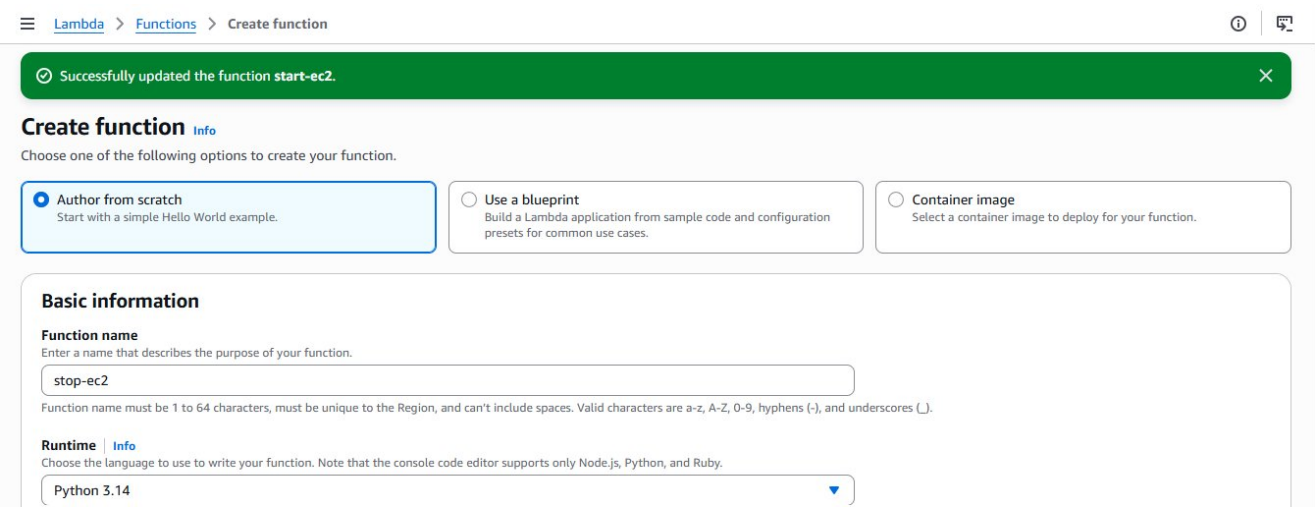
Both Lambda functions were created from scratch using Author from scratch with Python 3.14 as the runtime. start-ec2 was created first, followed by stop-ec2.




*Creating start-ec2 Lambda — Author from scratch, function name: start-ec2, Runtime: Python 3.14.*



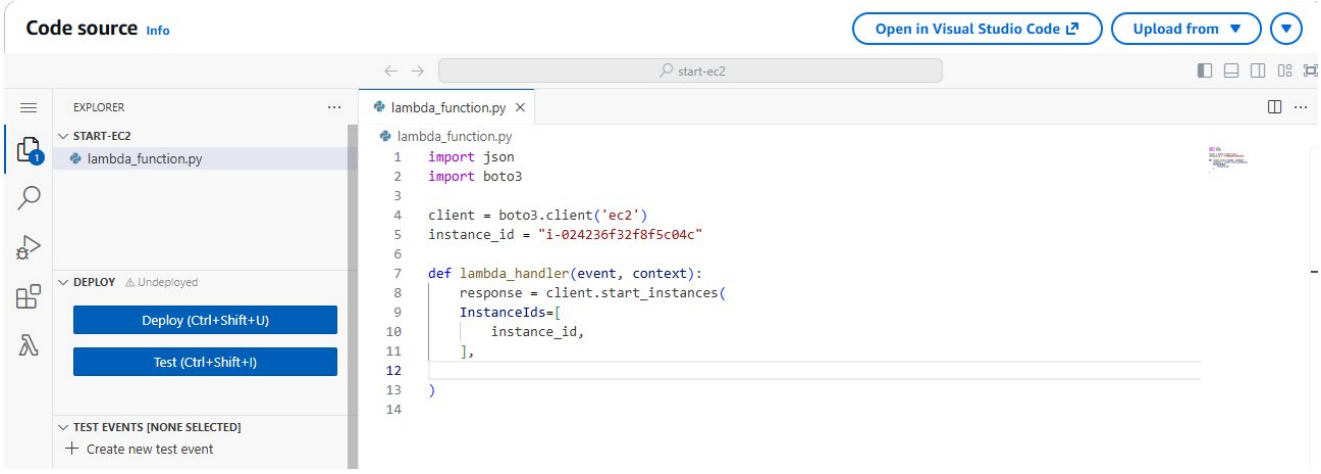

*start-ec2 function created successfully. Function ARN: arn:aws:lambda:eu-west-2:069062668071:function:start-ec2. Configuration tab open showing execution role.*



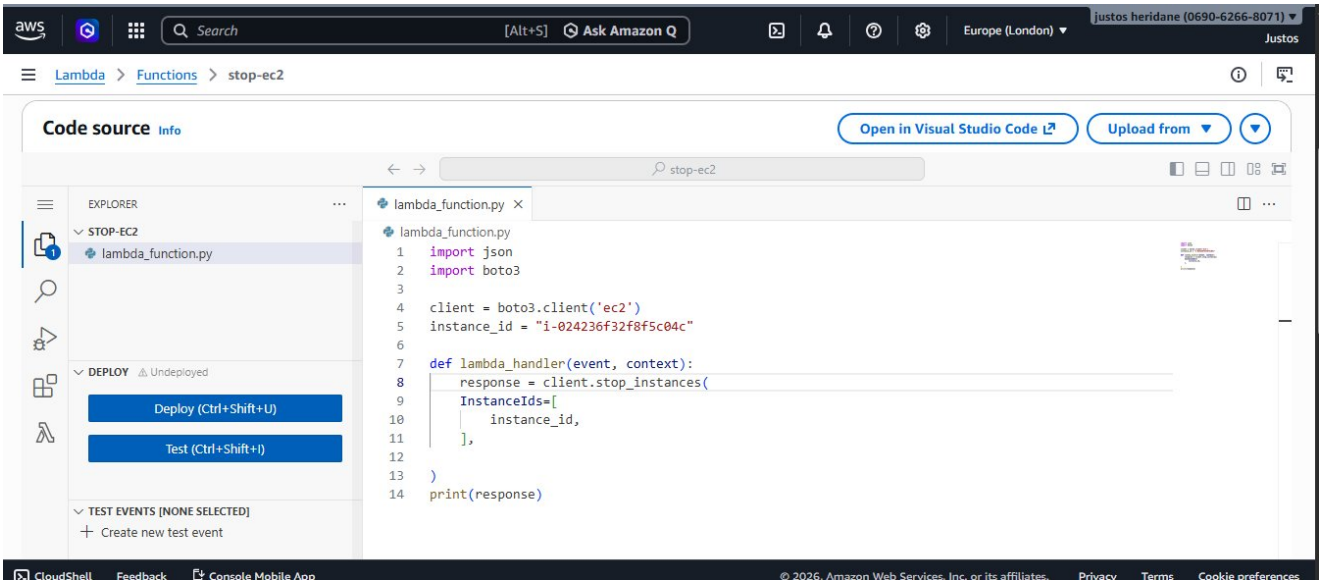

*Creating stop-ec2 Lambda — Author from scratch, function name: stop-ec2, Runtime: Python 3.14. Green banner confirms start-ec2 was already successfully updated.*

## 5 · Writing & Deploying the Lambda Code

The Lambda code was written directly in the AWS console code editor. Both functions follow the same pattern — create a Boto3 EC2 client, define the target instance ID, then call the appropriate API in the handler.




*start-ec2 Lambda code — import boto3, boto3.client('ec2'), start_instances() with instance ID hardcoded. Ready to deploy.*




*stop-ec2 Lambda code — identical structure but calling stop_instances(). Status: Undeployed, ready for Deploy button.*

### start-ec2 Code

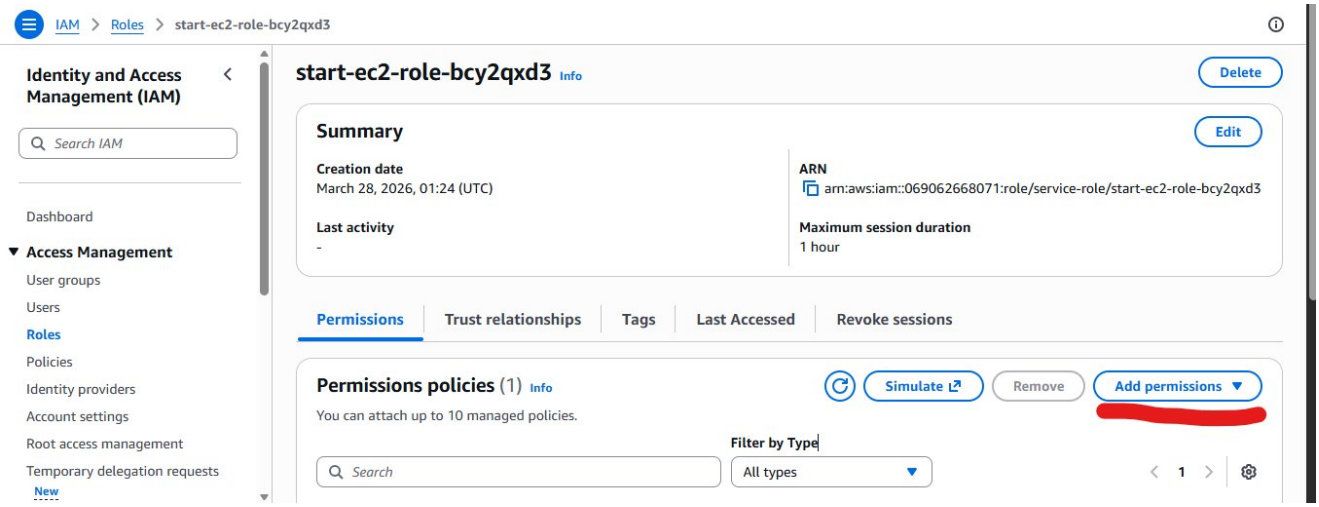
```
import json
import boto3

client = boto3.client('ec2')
instance_id = "i-024236f32f8f5c04c"

def lambda_handler(event, context):
    response = client.start_instances(
        InstanceIds=[instance_id]
    )
```

### stop-ec2 Code

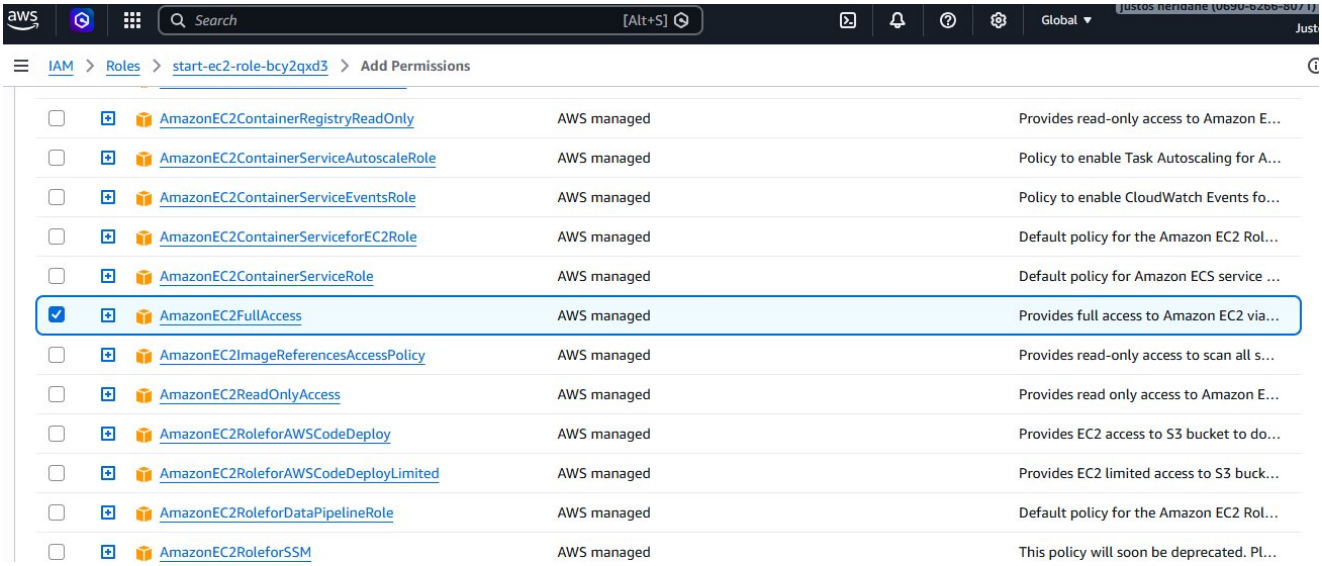
```
import json
import boto3

client = boto3.client('ec2')
instance_id = "i-024236f32f8f5c04c"

def lambda_handler(event, context):
    response = client.stop_instances(
        InstanceIds=[instance_id]
    )
```

# 6 · IAM Permissions

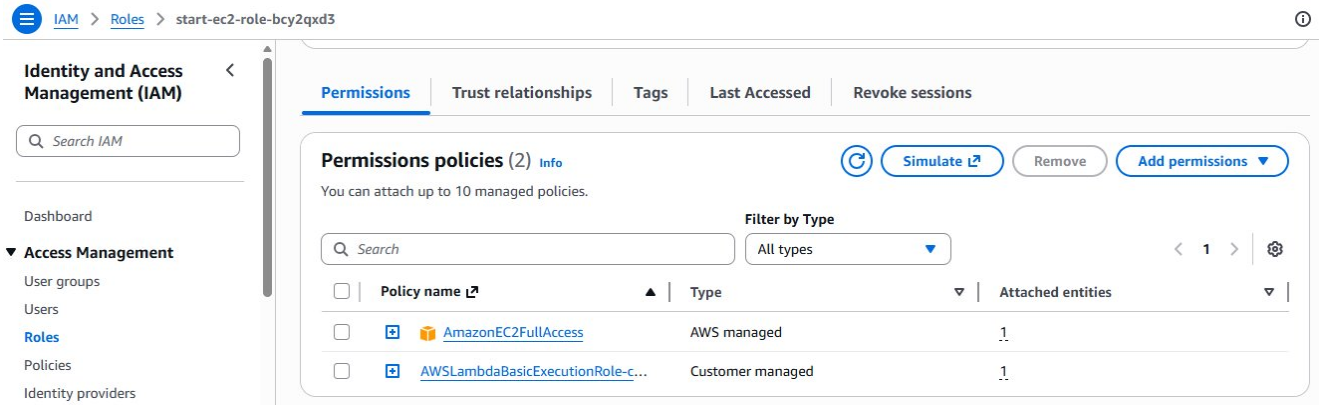
By default, a newly created Lambda function only has permission to write logs to CloudWatch. To allow it to control EC2 instances, AmazonEC2FullAccess was attached to the Lambda execution role.



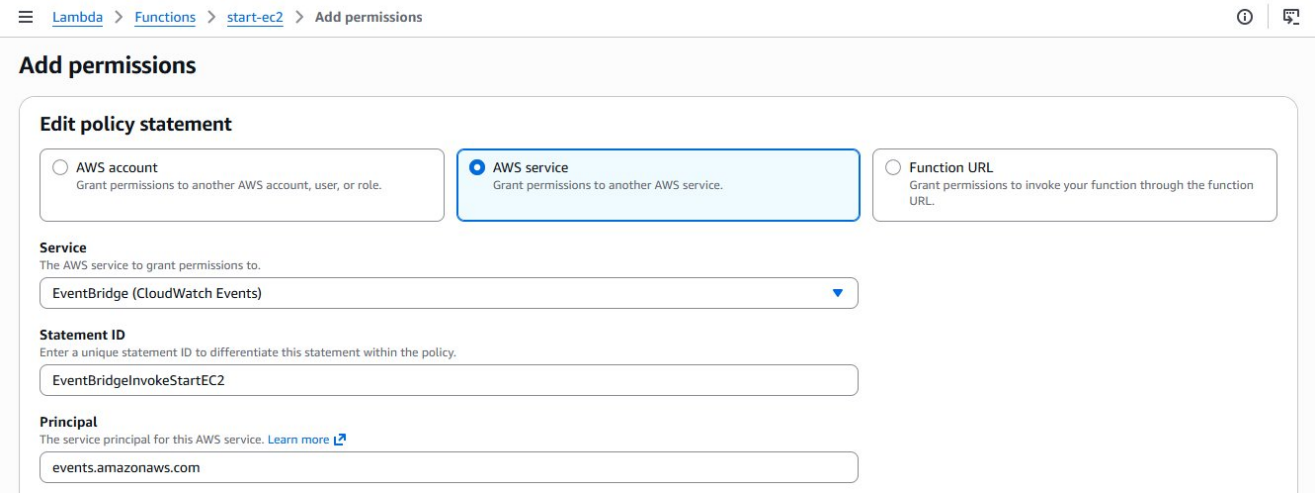

*IAM role start-ec2-role-bcy2qxd3 — Add Permissions page. Red arrow highlights the Add permissions button to attach policies.*




*Selecting AmazonEC2FullAccess from the AWS managed policy list — provides full access to Amazon EC2 via the AWS Management Console.*



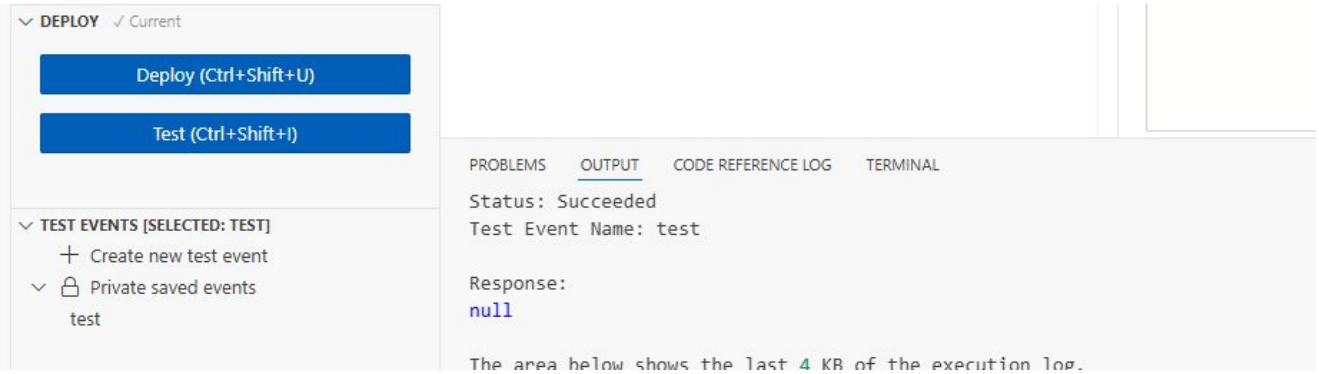

*Final permissions for start-ec2-role-bcy2qxd3 — two policies attached: AmazonEC2FullAccess (AWS managed) + AWSLambdaBasicExecutionRole (Customer managed).*




*Resource-based policy on start-ec2 Lambda — granting events.amazonaws.com (EventBridge) permission to invoke the function. Statement ID: EventBridgeInvokeStartEC2.*

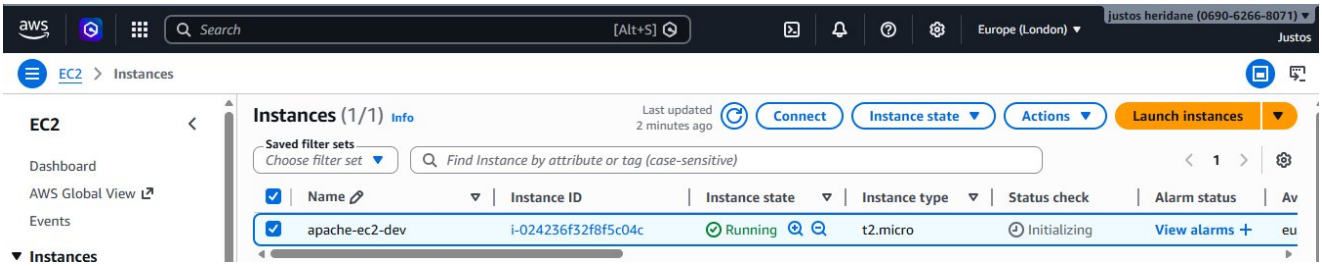
## 7 · Testing the Lambda Functions

Both Lambda functions were tested manually using test events in the AWS console before connecting EventBridge. This confirmed the Boto3 code was correct and the IAM permissions were working before setting up the automation.

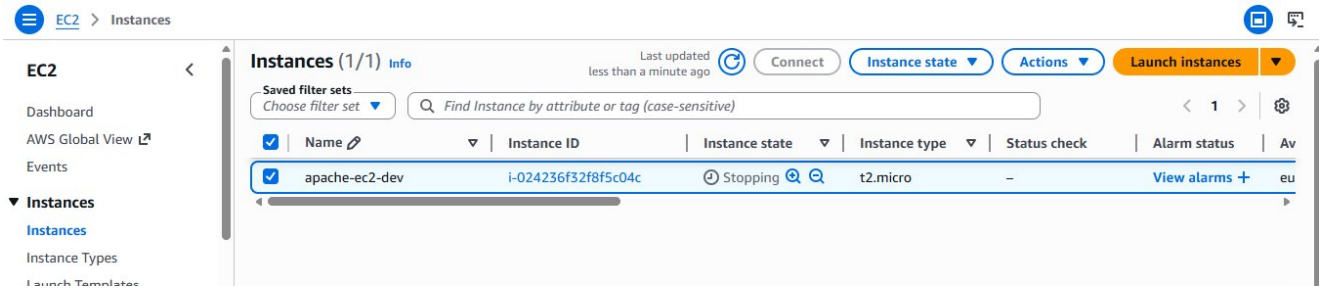



*start-ec2 test result — Status: Succeeded, Test Event Name: test. Response: null (expected — stop/start return no body by default).*

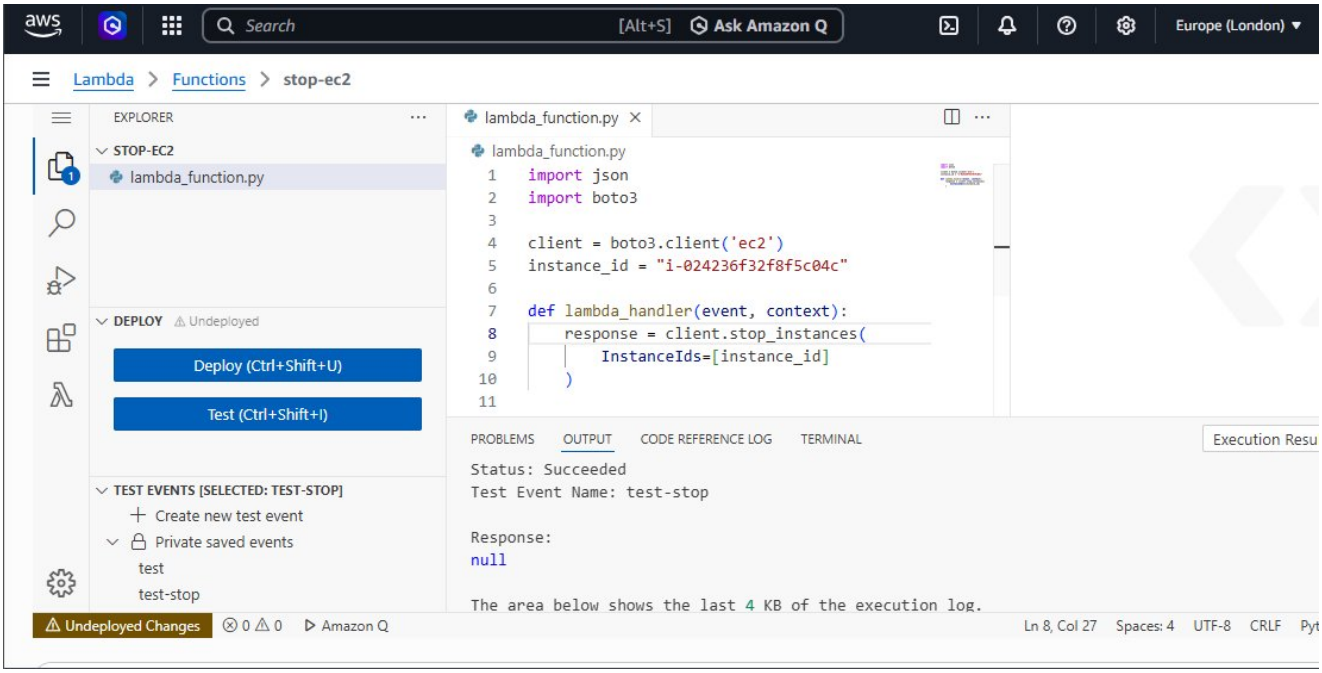
*Full execution log for start-ec2 — Duration: 920.49ms, Billed: 1490ms, Memory: 128MB, Max Used: 101MB, Init Duration: 568.92ms. Status: Succeeded.*




*EC2 console immediately after start-ec2 test — apache-ec2-dev now shows Running (Initializing). Confirms the Lambda successfully started the instance.*



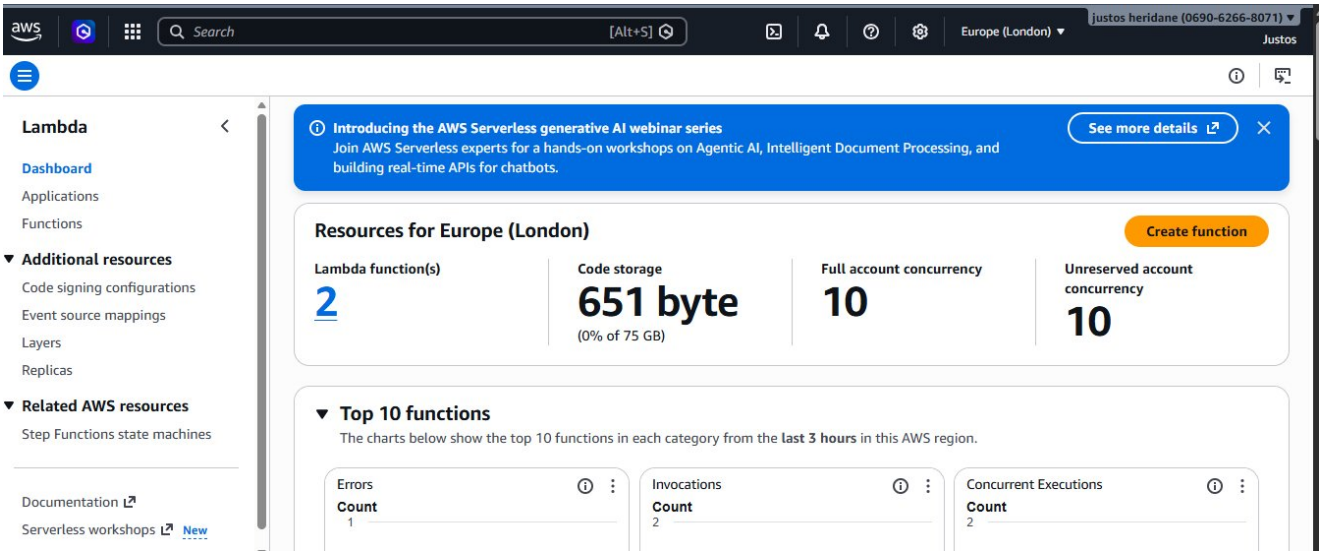

*stop-ec2 test — Status: Succeeded, Test Event Name: test-stop. EC2 instance transitions to Stopping state.*



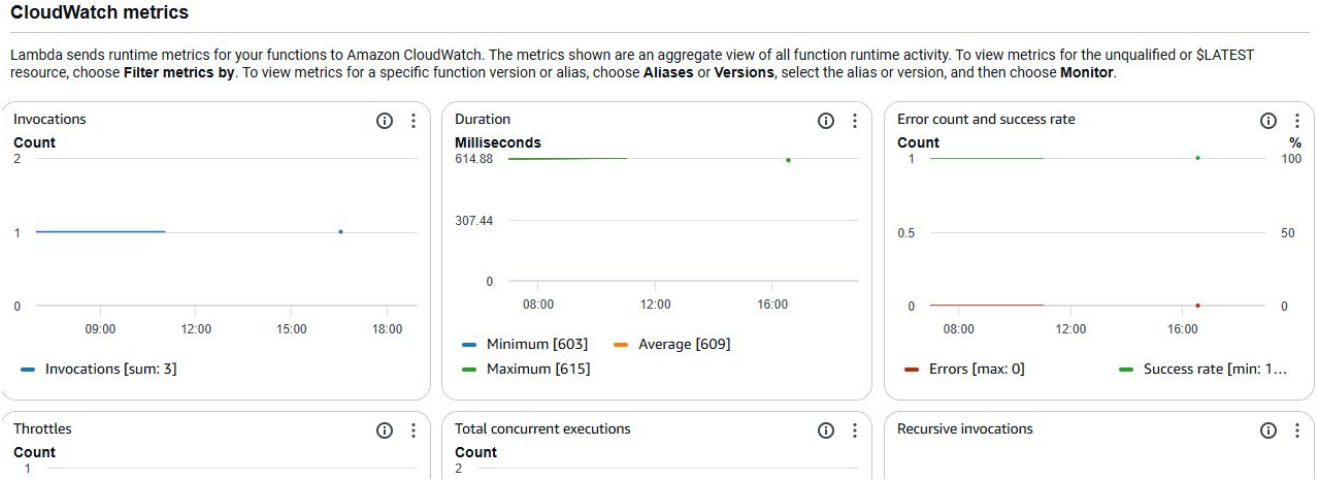

*EC2 console confirming Stopping state after stop-ec2 Lambda was invoked — the automation pipeline is working end-to-end.*

## 8 · CloudWatch Monitoring

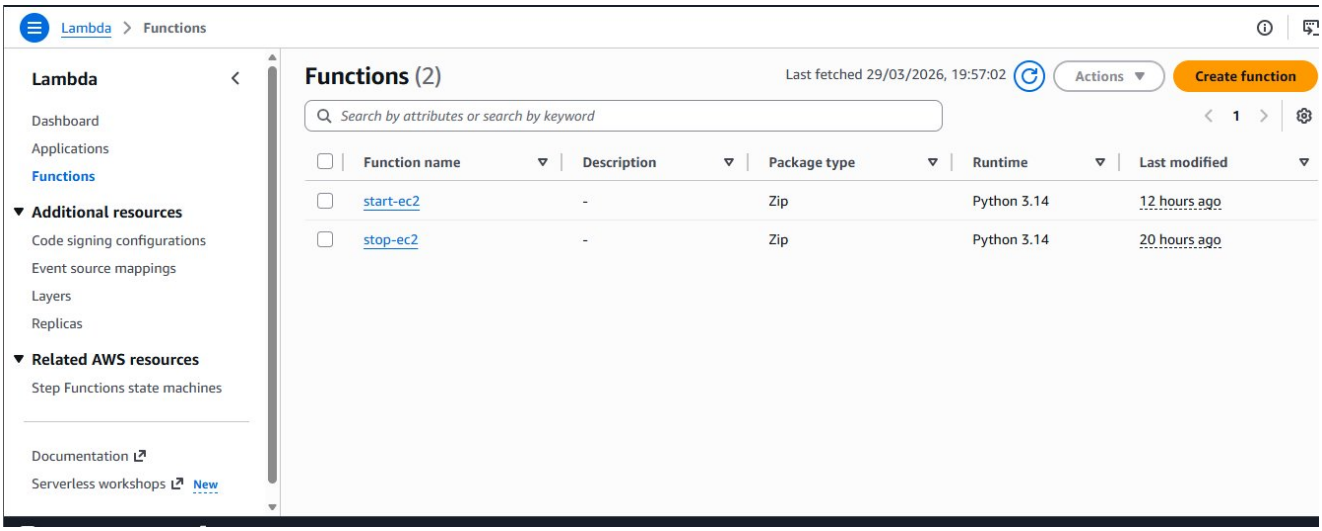
Lambda automatically sends runtime metrics to CloudWatch. The Monitor tab shows invocation counts, duration, error rate, and throttle data — useful for confirming the functions are running reliably and spotting any failures.




*Lambda dashboard — 2 functions deployed in Europe (London). 2 invocations counted, 651 bytes of code storage used.*



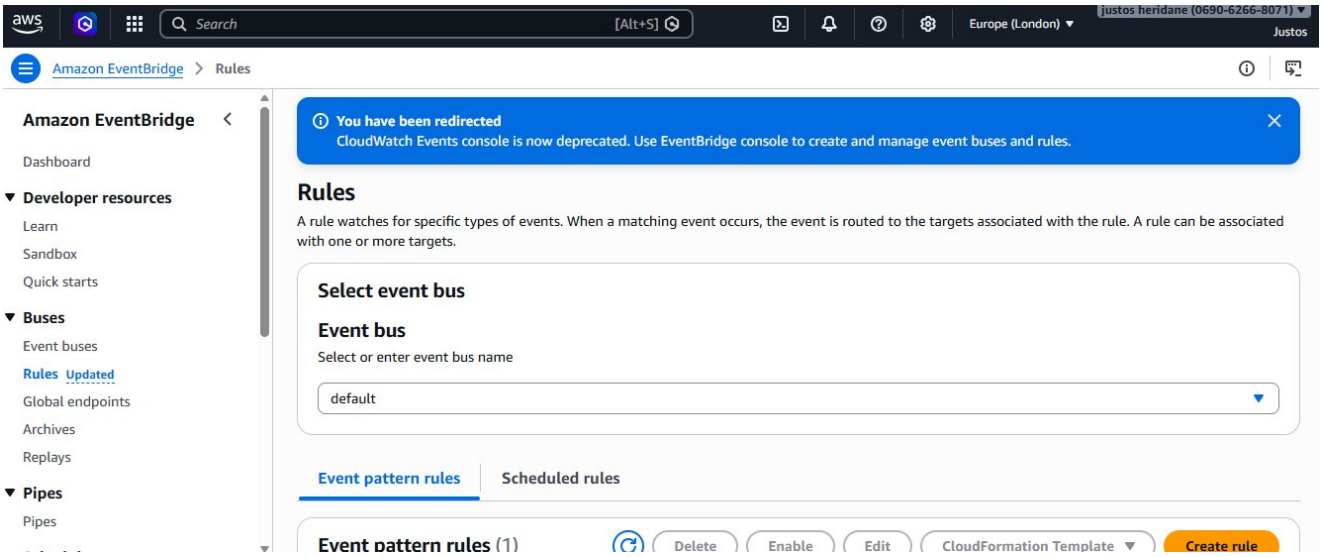

*CloudWatch metrics — 3 total invocations, average duration 609ms, max 615ms. Error count: 0. Success rate: 100%. No throttles recorded.*



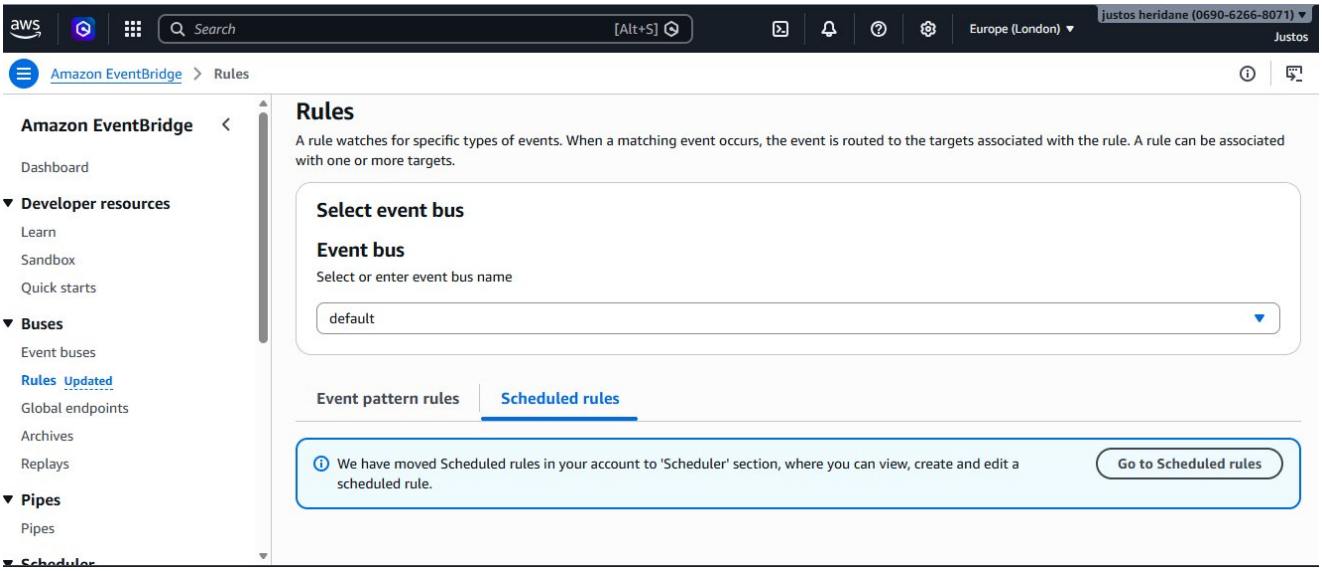

*Lambda functions list — start-ec2 and stop-ec2, both Python 3.14, Zip package type. start-ec2 last modified 12 hours ago, stop-ec2 20 hours ago.*

# 9 · EventBridge Scheduler — Cron Rules

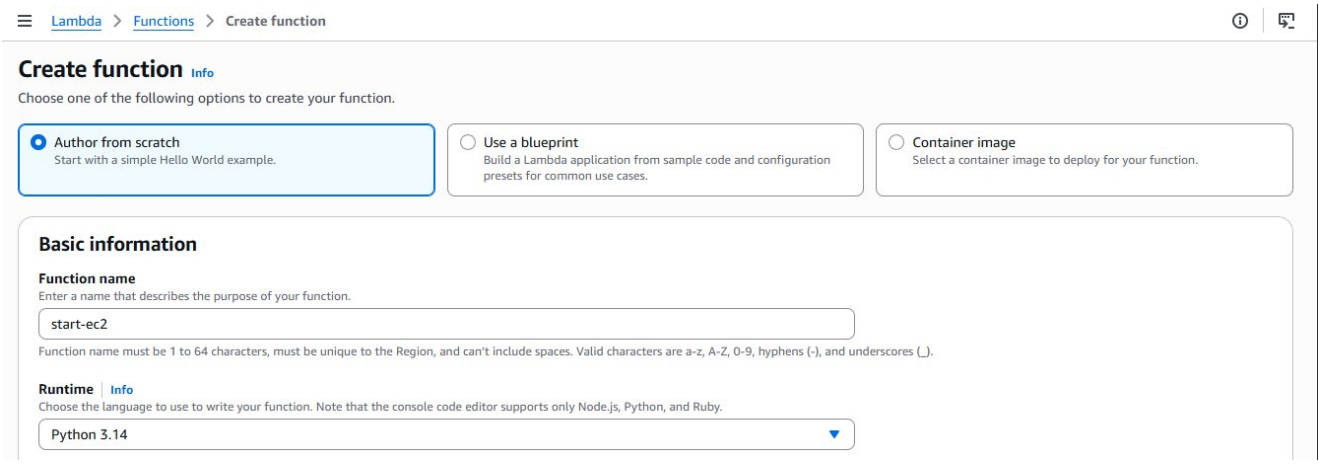
Amazon EventBridge Scheduled Rules provide the automation layer. Two rules were created with cron expressions to trigger the Lambda functions at the correct times Monday to Friday. AWS recently migrated scheduled rules from CloudWatch Events to the EventBridge Scheduler section.



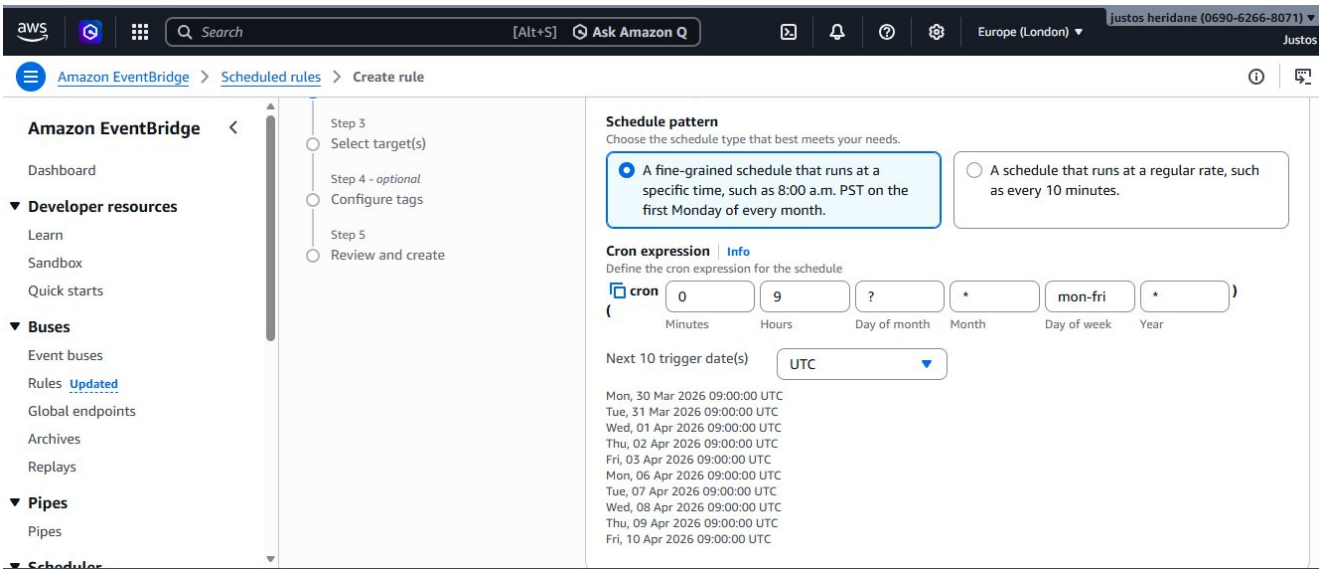

*AWS notification: CloudWatch Events console is deprecated — use EventBridge console. Scheduled rules are now managed under the Scheduler section.*



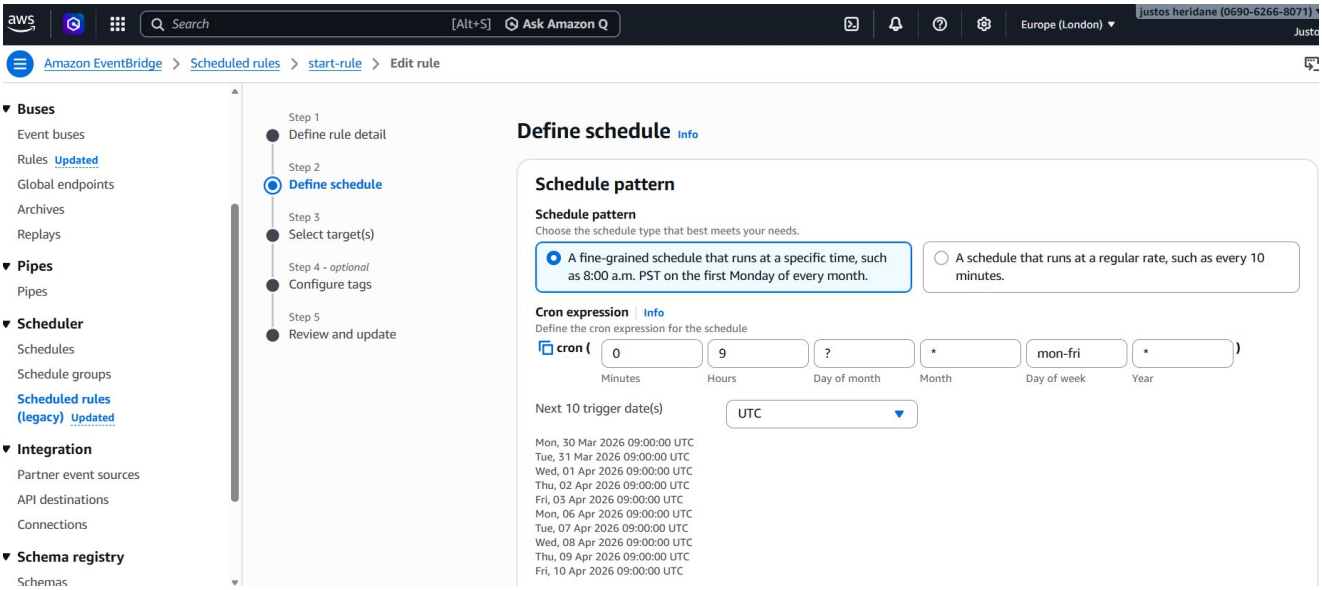

*EventBridge Rules page — Scheduled rules tab selected, default event bus. Starting point for creating the cron-based automation rules.*



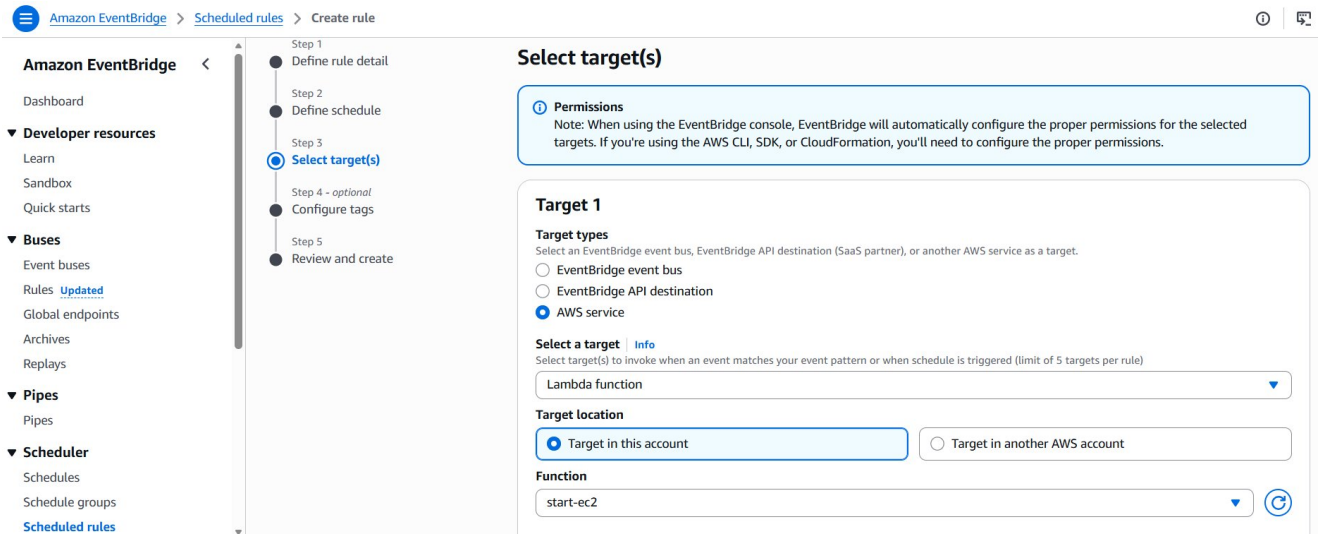

*EventBridge Create rule page — Step 3: Select target(s). AWS service selected, Lambda function as target type.*



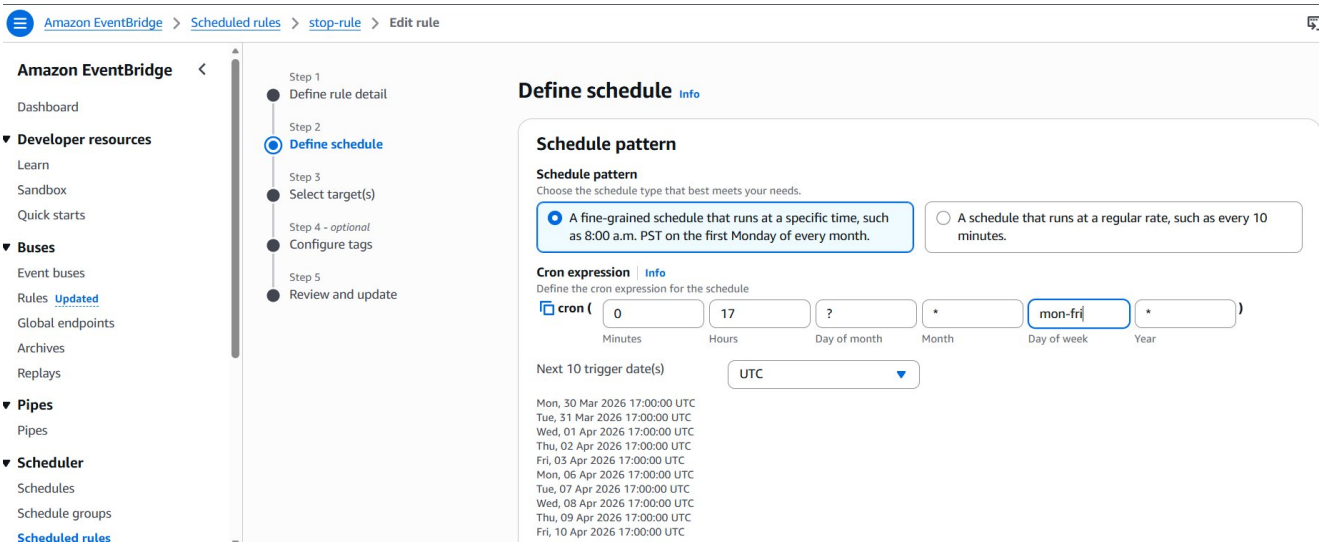

*start-rule cron setup — cron(0 9 ? * mon-fri *). Fine-grained schedule. Next 10 triggers all show 09:00 UTC on weekdays.*



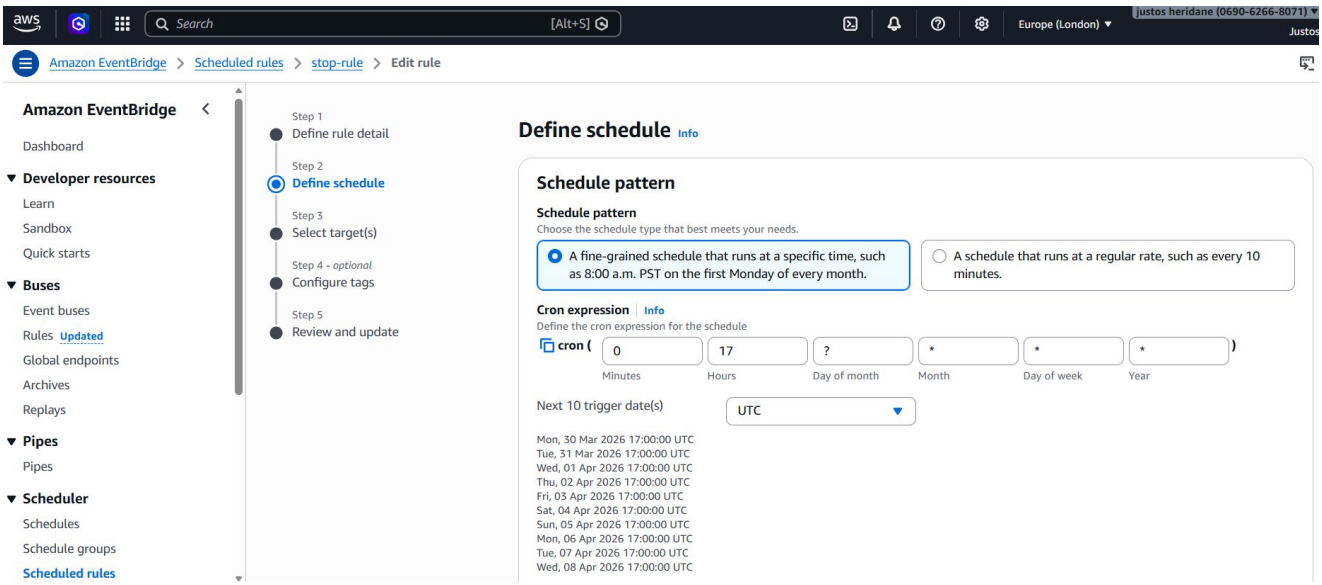

*start-rule edit view — cron(0 9 ? * mon-fri *) confirmed with Day of week: mon-fri. Trigger dates from Mon 30 Mar through Fri 10 Apr 2026, all at 09:00 UTC.*



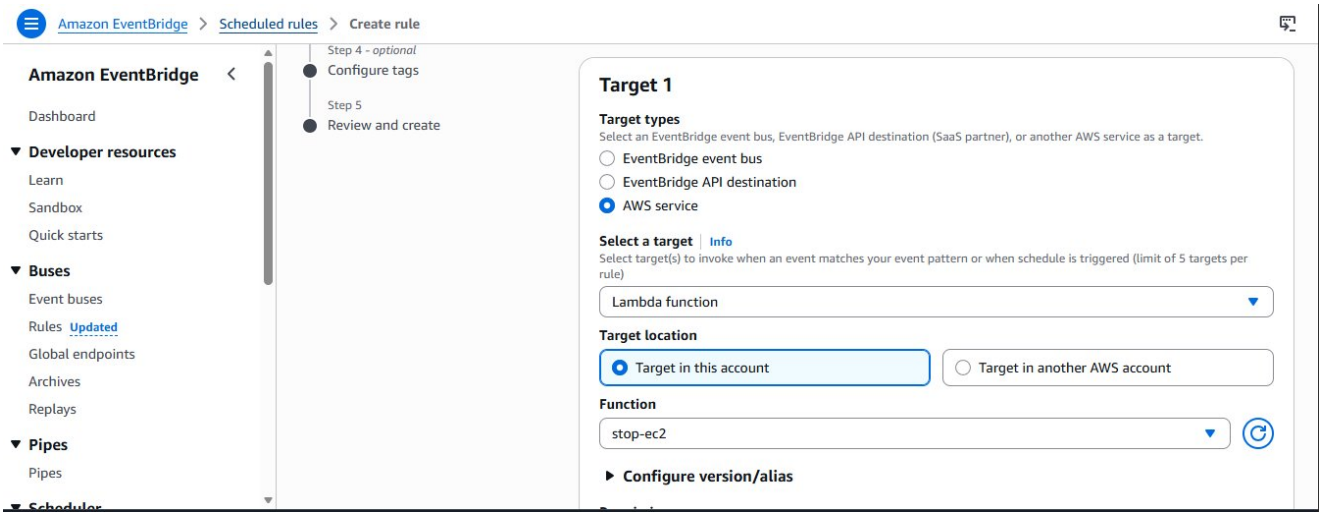

*start-rule target — AWS service → Lambda function → start-ec2. EventBridge automatically configures invoke permissions.*




*stop-rule cron — cron(0 17 ? * mon-fri *). Next 10 triggers show 17:00 UTC on weekdays.*



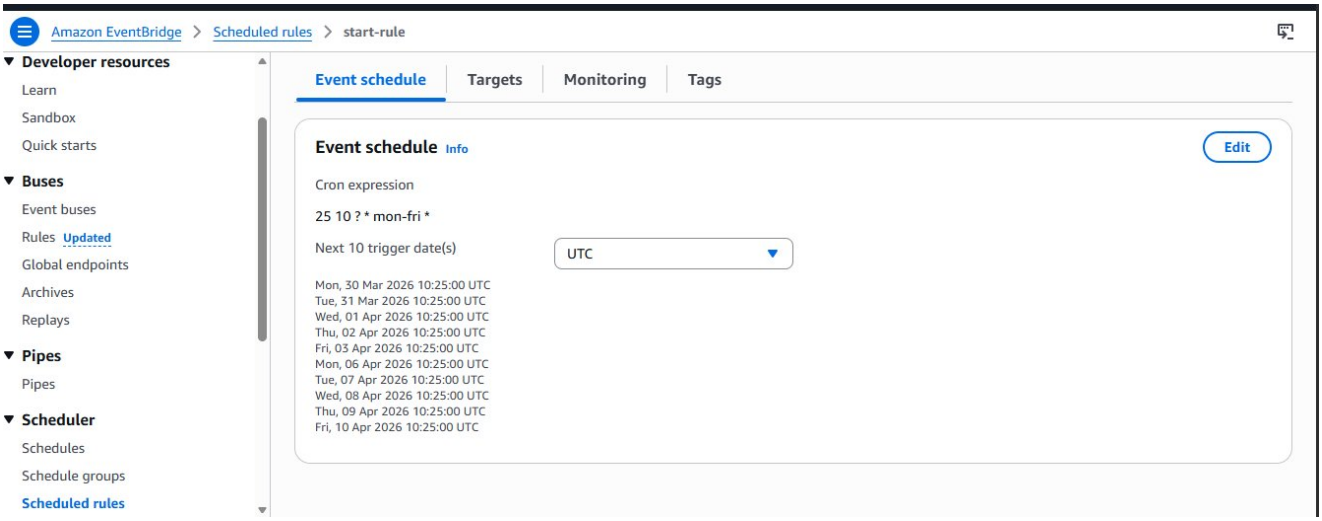

*stop-rule edit view — cron(0 17 ? * mon-fri *) confirmed. Triggers every weekday at 17:00 UTC.*



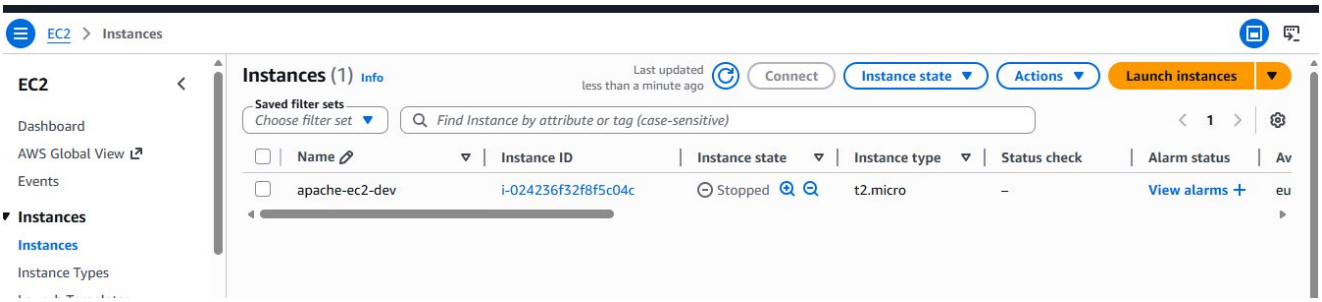

*stop-rule target — AWS service → Lambda function → stop-ec2. Target in this account selected.*

## 10 · End-to-End Scheduler Test

To confirm the full automation pipeline worked, the EventBridge start-rule was temporarily updated to trigger at the current time. This verified that EventBridge successfully invoked the Lambda function, which in turn started the EC2 instance automatically — no manual action required.




*start-rule event schedule showing cron(25 10 ? * mon-fri *) — time temporarily changed to current minute (10:25 UTC) to observe live trigger behaviour.*




*EC2 console after the scheduled stop-rule fired — apache-ec2-dev in Stopped state. Full end-to-end automation confirmed working perfectly.*

## 11 · Test Results Summary

| Test | Method | Outcome | Status |
|---|---|---|---|
| stop-ec2 manual test | Lambda test event | EC2 entered Stopping state | ✓ PASSED |
| start-ec2 manual test | Lambda test event | EC2 entered Running / Initializing | ✓ PASSED |
| IAM permissions | Policy attachment | EC2FullAccess + BasicExecution confirmed | ✓ PASSED |
| EventBridge stop-rule live trigger | Cron fired automatically | EC2 automatically stopped on schedule | ✓ PASSED |
| CloudWatch metrics | Monitor tab | 3 invocations, 0 errors, 100% success, avg 609ms | ✓ PASSED |

## 12 · Business Case & Interview Talking Points

### Business Case

EC2 instances are billed per hour of runtime. An instance running 24/7 accumulates ~730 hours/month. Restricting it to business hours (09:00–17:00 UTC, Mon–Fri) reduces this to ~200 hours/month — over 70% cost reduction. For larger fleets or bigger instance types, savings scale proportionally.

### Ideal Use Cases

- Dev/test environments — no need to run overnight or weekends
- Internal tools (Jenkins, dashboards) used only during business hours
- Training environments that must not be left running
- Any workload with a predictable, fixed operating schedule

### Complements Auto Scaling

Auto Scaling handles unpredictable load — scaling out during traffic spikes and back in when demand drops. The Instance Scheduler handles predictable on/off patterns — ensuring compute is completely off outside business hours. Together they provide full cost control across both axes.

### Skills Demonstrated

- Serverless compute — AWS Lambda, Python 3.14, Boto3
- Event-driven automation — EventBridge, cron expressions
- IAM — role creation, policy attachment, least-privilege
- Observability — CloudWatch metrics, invocation monitoring
- FinOps — cloud cost optimisation through automation

Interview Answer: "I built an EC2 Instance Scheduler to automatically stop and start instances on a business-hours schedule using Lambda and EventBridge. The business case was cost reduction — if instances only run during working hours, you cut compute costs by over 70% with zero manual effort. I also see it complementing Auto Scaling: the scheduler handles predictable on/off patterns while Auto Scaling handles unpredictable load within those hours. Together they give you complete cost control."